TECHNICAL REPORT · ARC PRIZE 2026 · ARC-AGI-3

Where the Points Go

A measurement study of an open-model agent on ARC-AGI-3

Lavin — Koris Lab · November 2026



Abstract

We report what limits an open-weight language-model agent on ARC-AGI-3 when it must play 110 unseen games in nine hours on one RTX Pro 6000. The agent builds on the public Duck harness (Tufa Labs), Daniel Franzen's Milestone-2 patch and lordhansolo's vLLM runtime for Qwen3.8-Flash-Next, a mixture-of-experts model with about 6B active parameters. Over 23 scored leaderboard submissions and six controlled two-hour runs on the 25 public games, we measured where score is lost.

Five findings stand out. **(1)** The score is token-bound: because later levels weigh more, about 10% more effective thinking per game is worth about 2.4 points on the hidden games. **(2)** Reading prompts costs about 15% of generation throughput, and on this hybrid model the serving cache never reaches the agent's previous reply, so every reply is read again on the next turn. **(3)** Tests on the public games are not valid for changes to what the agent reads: an adviser component that looked neutral in a leakage-controlled public test scored about 4 points lower on the hidden games in both of its submissions. **(4)** Levels the agent gets stuck on are mostly a lottery of its first reading: the same model cleared 26 of 41 such levels in other runs, while search could not rescue them even when given the goal. **(5)** Two mechanical changes to the token economy, an effort ladder after truncated replies and lighter history trims, raised levels cleared per token

by 4 to 26% in a controlled run. We release the agent and our measurement scripts.

1 Introduction

ARC-AGI-3 replaces static puzzles with interactive games. The agent sees a 64×64 grid in 16 colours, has a handful of actions, and has to find out from experience what each level asks for. Each level scores $(h/a)^2$ against the action count h of the median first-time human, capped at 115%; later levels weigh more, and a game's score cannot exceed the weighted share of the levels cleared (ARC Prize methodology).

The Kaggle competition adds hard limits: no internet, open models only, one 96 GB GPU and nine hours for 110 games, of which 55 are scored on the public leaderboard and 55 decide the final ranking. Within these limits most competitive entries we know of run the same open model family inside the Duck harness, yet public scores range from about 25 to above 70 (leaderboard snapshot, 25 October).

We did not find an architecture that closes that gap. This paper is a careful account of where an agent in this family loses its points, with numbers other teams can check against their own logs. Some losses are mechanical and cheap to remove. Others are traps that cost us leaderboard submissions to find, and we report them so others need not.

2 Related work

Harnesses. Tufa Labs' Duck harness exposes the game as Python variables in a REPL, shows the board as an image, an ASCII grid and segmentations, and evicts old messages so play can continue indefinitely; it won Milestone 1 (Tufa Labs). Franzen's Milestone-2 notebook adds agent tools, a slot scheduler and faster serving on Qwen3.8-Flash-Next (Franzen).

World-model agents. A coding agent that keeps an executable model of the game, verifies it against recorded transitions and plans with it reaches 41 to 58% RHAE on the public games with GPT-5.4 and GPT-5.5 (Rodionov, 2026); a

follow-up ablation finds that model capability and reasoning effort matter more than the design variants (Rodionov, 2026b). Tycho (Lehmann et al., 2026) and OPINE-World (Courtis et al., 2026) report 78 to 100 on the public games with frontier models. None of these was evaluated on held-out games, and none can run offline on an open model.

Exploration. Training-free graph exploration solved 30 of 52 levels of the six-game preview (Rudakov et al., 2025). The preview winner, a convolutional action model, fell to near zero on the full benchmark once efficiency was scored (ARC Prize). Liew shows that the public games can be cleared by trivial strategies and argues that only held-out games test exploration (Liew, 2026).

Serving. Speculative decoding pays less as batches grow and decoding becomes compute-bound (Scaling LLM speculative decoding, 2025). Prefix caching for hybrid attention models stores recurrent state only at block boundaries inside prompts; newer vLLM modes also store it while decoding (vLLM #36649, #50172).

3 The agent

Each turn the agent receives the board as an image and as an ASCII grid, an image of the cells that changed, the harness's summary of the last step and its own retained Python functions. It thinks in Qwen3.8's thinking mode (maximum effort, temperature 0.7) and calls a Python tool that can execute one or more game actions. History is kept up to 104K tokens; when it fills, old messages are dropped with hysteresis, so that trims are rare and large.

Up to 18 games hold a decoding slot at a time. A priority gate (affectify's D′) gives a freed slot to the waiting game with the highest expected value, and slots change hands at history trims, the one moment when the cached prefix is lost anyway. Four add-ons run on top of the harness: a memory of the agent's own rule for each cleared level, measured facts about progress after 40K tokens on a level, a second opinion from a fresh model instance on a stalled level, and a pool of rules learned on other games of the same run.

Serving uses lordhansolo's vLLM 0.29 build with a mixed NVFP4/FP8 checkpoint of Qwen3.8-Flash-Next (about 125B parameters in total, about 6B

active per token), 448 of 512 experts kept by REAP pruning, and three draft tokens from the model's multi-token-prediction head. At 18 concurrent requests it generates about 1,000 tokens per second.

4 Methods

- **Leaderboard noise.** We count one submission as one draw. Identical code scored 32.76 and 24.03 (v36) and 32.03 and 26.96 (v48); across 23 submissions the per-run standard deviation is about 2.7 points, so single submissions cannot separate versions closer than about 5 points.
- **Public-set runs.** Two-hour runs of all 25 public games (121 minutes, the competition's GPU time per game) and 30-minute health checks.
- **Same-game token matching.** For every game we take the cumulative generated tokens at each level clear. At budget B we count levels cleared within B tokens on the games the reference run used at least B tokens on; a game a run finished counts with all its levels, and a game a run stopped on before B is left out. This removes differences in machine speed and scheduling.
- **Leakage control.** Our advisers saw a design catalogue that described every public game. In a clean test the catalogue line for the game being played is removed from every adviser request (228 of 228 requests in our clean run).
- **Serving logs.** vLLM's 10-second statistics (prompt and generation throughput, running requests, per-position draft acceptance) and per-request token usage (prompt, cached, completion).

5 Findings

5.1 The score is token-bound

When a level is cleared at or below the human action count, which holds for most of our cleared levels (median 0.56 to 0.74 times the human count), clearing the first K levels of a seven-level game scores K(K+1)/2 out of 28. Our leaderboard level of about 29 corresponds to about 3.5 levels per hidden game; 37 needs about 4.1.

On the public set, raising the budget per game from 150K to 200K tokens (+33%) added 15% more levels, and from 200K to the full budget of about 260K (+30%) another 14%: an elasticity of about 0.45. Through the quadratic weights, 10% more effective tokens is worth about 9% more score, or about 2.4 points at our level. A change that saves 10% of tokens is therefore worth as much as a new idea that raises per-level success by several points.

5.2 A hidden serving tax

Across about 1,000 ten-second windows of two full runs, generation throughput falls linearly with prompt throughput ($r = -0.80$): about 1,125 tokens per second when little is being read, against about 970 on average. Reading prompts costs about 15% of generation.

Table 1. Where prompt reading comes from (uncached prompt tokens of player requests, two-hour runs v47 and v49). The split of the first two rows comes from the per-request fit described below.

Source	Share of reading	Generation cost
New content each turn (about 1,500 tokens per request)	≈23%	≈3.5%
The previous reply, read again (about 1.07 times its length)	≈35%	≈5.4%
History trims (about 160 per run, median about 49K tokens each)	42%	≈6.5%

The middle row is the surprise. In 97% of requests the cache stops 128 to 2,048 tokens before the end of the previous prompt (median 188) and never reaches into the reply. The uncached part of a request is about 1,500 tokens plus 1.07 to 1.10 times the length of the previous reply. The model's recurrent (GDN) state is saved only at block boundaries inside prompts, so the tokens the agent generated on its last turn can never be reused. A cache mode that saves state while decoding should remove most of this cost; we could not test it in our runtime before the deadline.

Decoding is also compute-bound at our batch size: per-request speed falls from 90 tokens per second with 10 requests running to 63 with 18. The three draft tokens are accepted at rates of 0.755, 0.550 and 0.393 (median of 2,327 windows). A step-time model fitted to these numbers predicts that two draft

tokens would be about 6% faster than three. Speculative decoding is exact, so this change could not alter the agent's behaviour. We report it as a prediction, not a result.

5.3 Public tests mislead for changes to what the agent reads

We added "first impressions": at the first turn of every level, two fresh instances of the model look at the new board, the controls and the rules of earlier levels and propose up to three goal hypotheses with the cheapest test for each. On public games whose descriptions the advisers had not seen, they named the right goal in every level we graded by hand, and the player cited them in about 21% of the turns that received them.

Table 2. The same component under three evaluations.

Evaluation	Result
30-minute public test, catalogue not hidden	about 20% more levels per token than earlier runs at 80K tokens
Two-hour public test, catalogue hidden (same 25 games, token-matched)	10 to 17% fewer levels at 40K to 80K tokens; equal at 120K to 200K
Hidden games (leaderboard)	24.0 and 24.21, against a mean of 28.54 over our other 21 submissions

If the component were neutral, two scores this low would occur by chance about 0.3% of the time. An earlier component that pointed out matching shapes on each level's first board scored 47 against 55 to 57 in public tests and was never submitted. Our reading: components that the agent trusts shape its first hypothesis, and on unseen games a plausible but wrong hypothesis costs more than it saves. Public games, which our components and their authors have seen, cannot reveal this. Any change to what the agent reads needs a held-out game set; leaderboard runs, at about ±2.7 points each and one per day, are too slow to serve as one.

5.4 Stuck levels are a lottery, and search cannot fix them

We collected the 41 levels on which a run spent at least 60K tokens without clearing them. The same model cleared 26 of them in another run, often within 9 to 21 moves. In 25 of those 26 cases the stuck run had already used the

clearing run's main interaction and seen its effect: it knew the controls and misread the goal.

Restarting does not help. A level stalled at 60K tokens was cleared within the next 60K in 62% of cases when the agent continued, against 44% for a fresh start; at 100K the figures were 51% and 35%. Search does not help either: three structured search engines with learned interface masks cleared 3 of 40 stuck levels within 12,000 moves, and only 4 of 25 even when given the true final board as the goal.

5.5 A cheaper token economy

Two mechanical changes target findings 5.1 and 5.2. The *effort ladder* is a built-in harness option, off by default: after a reply is cut off at the 12,288-token cap, the following requests think at "medium" effort until an action changes the board. Our two-hour runs had 33 to 41 such cut-offs each, holding about 7% of all tokens. *Lighter trims* keep about 35K tokens after a trim instead of half the budget (about 52K), which halves the history read again at each trim and lowers the average history depth by about 12%. A scheduler simulation of the longer slot sessions this causes predicted a cost of 0.06 to 0.20 points, with every game still started.

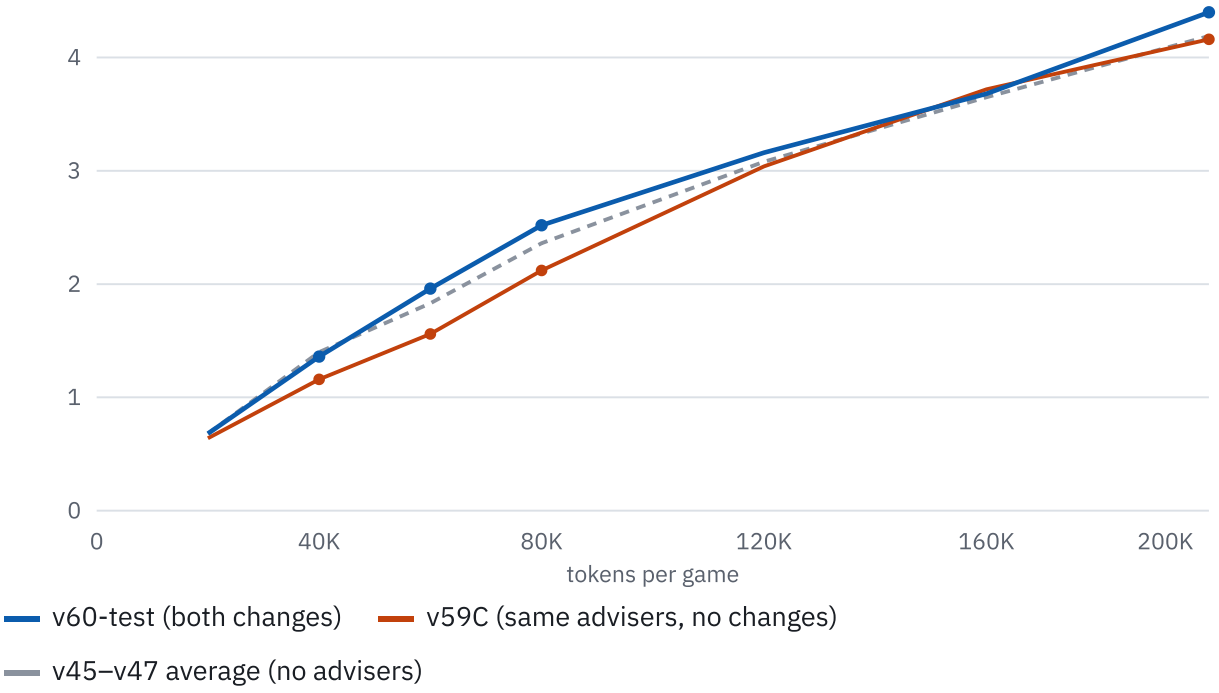


Figure 1. Levels cleared per game within a token budget, same 25 public games, two-hour runs. Both v60-test and v59C ran with the advisers and the catalogue hidden; v60-test also ran without the per-

turn change image.

Against v59C, which used the same advisers in the same clean setting, v60-test cleared 17%, 26% and 19% more levels at 40K, 60K and 80K tokens, about the same at 120K and 160K, and 6% more at 200K. It reached 59.39 with 131 levels, our best two-hour run. The effort ladder fired 120 times, cut-off replies fell from 50 to 38, and the agent took 10% more actions per thousand tokens. Total throughput did not rise: that machine was about 3% slower, and more, shorter turns increased prompt reading. The gain came as more levels per token, not as more tokens. This was one run with three simultaneous changes, so we cannot attribute the gain to each of them.

6 Results

Table 3. Leaderboard (55 hidden games) and two-hour public-set scores of the versions discussed.

Version	What changed	Leaderboard	Public, 2 h
v36	Franzen base, vLLM serving, add-ons	32.76, 24.03	—
v45–v47	pruned model, more slots, cross-game rules	26.68, 29.34, —	55.2–56.8
v48	v47 with a later search trigger	32.03, 26.96	—
v49	shape-relation hints (not submitted)	—	47.32
v55, v59b	first impressions	24.0, 24.21	55.71 (clean)
v60	v48's player, effort ladder, lighter trims	33.39	59.39*

* Measured as v60-test, which also had the advisers on and no change image. The submitted v60 has neither, following finding 5.3.

v60's first submission scored 33.39, our best leaderboard result, about 3.5 points above the average of our earlier submissions. One submission carries about ±2.7 points of noise, so this is consistent with, but does not yet establish, the gain measured in section 5.5.

7 Limitations

Most comparisons rest on one run per configuration. Token matching removes machine speed and scheduling but not the run-to-run variance of the agent itself. Our two-hour comparisons use the public games, which finding 5.3 shows can mislead; we rely on them only for changes that do not alter what the agent reads. The elasticity behind finding 5.1 is measured on the public games and extrapolated to the hidden ones. The serving findings are specific to a hybrid mixture-of-experts model in one vLLM build.

8 Lessons

1. **Count tokens before ideas.** At the frontier of this benchmark the score is quadratic in levels and levels are concave in tokens, so cheap throughput gains compete with expensive reasoning gains.
2. **Audit the serving stack.** Measure prompt reading against generation, check whether the cache reaches the previous reply, and choose the draft length for the batch size you actually run.
3. **Build a held-out set before changing what the agent reads.** Public games and their descriptions leak into components and into their authors' judgement; the leaderboard is too noisy to replace a held-out set.
4. **Invest in first readings, not in rescue.** Stuck levels are mostly lost to an early wrong reading; restarts and search both did worse than continuing.

9 Code and data

The submitted Kaggle notebook (v60) will be public under an open-source licence after 2 November 2026: `[link pending]`. The measurement scripts used here (token matching from `benchmark.json`, serving-log fits, cache-boundary analysis, stuck-level replays and the scheduler simulation) will be released with it.

10 Acknowledgements

We build directly on Tufa Labs' Duck harness, Daniel Franzen's Milestone-2 solution, lordhansolo's vLLM runtime and checkpoint, and affectify's D′ scheduler. We thank the ARC Prize team for the benchmark. This report was

prepared with the help of an AI assistant (Claude); every number in it comes from our own runs and logs.

References

1. ARC Prize Foundation. ARC-AGI-3 methodology. docs.arcprize.org/methodology
2. Tufa Labs. The Duck harness (Milestone 1). x.com/tufalabs
3. D. Franzen. ARC-AGI-3 Milestone 2 solution. kaggle.com/code/dfranzen
4. S. Rodionov. Executable World Models for ARC-AGI-3 in the Era of Coding Agents. arXiv:2605.05138
5. S. Rodionov. Do Coding Agents Need Executable World Models, Simplification, and Verification to Solve ARC-AGI-3? arXiv:2607.15439
6. Lehmann, Aioanei, Vahdati. Tycho: Active Abstraction with Programmatic World Models for ARC-AGI-3. arXiv:2607.28287
7. Courtis, Li, Sanner. OPINE-World. arXiv:2607.01531
8. Liew K. H. Explore Before You Solve: The Speed–Depth Trade-off in Epistemic Agents for ARC-AGI-3. arXiv:2605.25931
9. Rudakov, Shock, Cowley. Graph-based exploration for ARC-AGI-3 interactive reasoning tasks. arXiv:2512.24156
10. ARC Prize. ARC-AGI-3 Preview Agent Competition. arcprize.org
11. Scaling LLM speculative decoding. arXiv:2511.20340
12. vLLM project. Prefix caching for hybrid GDN models. PR #36649, PR #50172